

Executable code

Run JavaScript, TypeScript, Python, and R blocks at build time and embed their output.

Shravan Goswami / <https://shravangoswami.com>

Almanac can run a fenced code block during the build and put its output on the page. This is opt-in, because executing code is a bigger commitment than rendering it.

Turn it on in `astro.config.mjs`:

```
almanac({
  title: "My Project",
  future: { execute: true },
})
```

Then mark a block with `exec`:

```
```js exec
const rows = [1, 2, 3].map((n) => n ** 3);
console.log(rows.join(", "));
```
```

Which renders the source followed by its result:

```
const rows = [1, 2, 3].map((n) => n ** 3);
console.log(rows.join(", "));

1, 8, 27
```

TypeScript works too, with types stripped before running:

```
interface Point {
  x: number;
  y: number;
}

const distance = (a: Point, b: Point): number =>
  Math.hypot(b.x - a.x, b.y - a.y);

console.log(distance({ x: 0, y: 0 }, { x: 3, y: 4 }));

5
```

Errors do not break the build

A block that throws reports the failure in place and the build carries on, so one broken example cannot stop you shipping:

```
JSON.parse("{ not valid json }");
```

```
<anonymous_script>:1
{ not valid json }
^
```

```
SyntaxError: Expected property name or '}' in JSON at position 2 (line 1 column 3)
at JSON.parse (<anonymous>)
at file:///almanac/block.mjs:1:6
at ModuleJob.run (node:internal/modules/esm/module_job:343:25)
at async onImport.tracePromise.__proto__ (node:internal/modules/esm/loader:681:26)
at async asyncRunEntryPointWithESMLoader (node:internal/modules/run_main:117:5)
Node.js v22.23.1
exited with code 1
```

Options

Add these to the fence after exec:

| Option | What it does |
|--------------|---|
| hide-code | Show only the output |
| hide-output | Run the block but render nothing |
| timeout=5000 | Override the 30 second default, in milliseconds |

A block that only sets something up can hide its output:

```
globalThis.__unused = "this block runs but shows nothing";
```

Caching

Results are cached in `.almanac/exec/`, keyed by a hash of the code, the language, the runtime version, and any declared dependencies. An unchanged block is never run twice, and a Node upgrade invalidates every entry, because output that depends on the runtime should not survive it.

Commit the cache directory to make CI builds reproducible and fast, or leave it out of version control and let CI cache it.

Python

Python runs too, through [Pyodide](#). It is an optional peer dependency, so install it only in projects that want it:

```
npm install pyodide
```

Then a python block behaves like any other:

```
```python exec
import statistics

samples = [12, 15, 9, 22, 17]
print("mean: ", statistics.mean(samples))
print("stdev: ", round(statistics.stdev(samples), 3))
```

import statistics

samples = [12, 15, 9, 22, 17]
print("mean: ", statistics.mean(samples))
print("stdev: ", round(statistics.stdev(samples), 3))

mean:    15
stdev:   4.95
```

The whole Python standard library is available. Third party packages are not installed for you yet, so numpy and friends need Pyodide's own package loading, which Almanac does not drive.

Pyodide starts in about a second, and it starts once per block that actually runs. Cached blocks cost nothing, so this shows up on the first build and then mostly disappears.

Python runs in its own process for the same reason JavaScript does, and it matters more here: a WASM runtime cannot be interrupted from the thread it runs on, so an infinite loop would hang the build rather than time out.

R

R runs through [WebR](#), also an optional peer:

```
npm install webr
```

```
samples <- c(12, 15, 9, 22, 17)
cat("mean: ", mean(samples), "\n")
```

```
cat("sd:  ", round(sd(samples), 3), "\n")
print(summary(samples))

mean:  15
sd:    4.95
Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 9       12       15      15      17      22
```

Autoprint works the way it does in an R console, so a bare expression prints its value. Warnings appear as warnings and do not fail the block; a `stop()` is reported as an error.

WebR starts faster than Pyodide, around half a second. Installing R packages from a repository is not wired up, so the base R distribution is what you get.

Jupyter kernels

Pyodide and WebR are convenient because they need no setup, and limited for the same reason: you get the base distribution and nothing you installed yourself. When you need a real environment, name a kernel:

```
```python exec kernel=python3
import sys
print("platform:", sys.platform)
```
```

Naming a kernel is what routes the block to a Jupyter server, so a plain `python exec` keeps running locally under Pyodide. The two can sit on the same page.

Point Almanac at the server through the environment:

```
export ALMANAC_JUPYTER_URL=http://127.0.0.1:8888
export ALMANAC_JUPYTER_TOKEN=your-token
```

The token is read from the environment rather than the config file on purpose: a credential in `astro.config.mjs` is a credential in version control.

No extra dependency is needed. Almanac talks to the server's REST and WebSocket API directly, creating a session per block and deleting it afterwards so a build cannot leave kernels running.

If the server is not configured or not reachable, the block reports that where its output would go and the rest of the site builds. That failure is never cached, so fixing the server and rebuilding is enough.

Rich output

A kernel can return more than text, and Almanac keeps it:

| Kernel output | Rendered as |
|------------------|------------------------------------|
| image/png | an inline <code></code> |
| image/svg+xml | inline SVG |
| text/html | the markup, as given |
| application/json | a formatted block |

So a matplotlib figure or a pandas table arrives on the page as a figure or a table. The plain text alternative is kept alongside it, so the output still reads sensibly where an image cannot be shown.

Local runtimes emit text only today. The artifact plumbing is shared, so a Pyodide figure is a matter of teaching that driver to capture one.

What is not here yet

Installing packages into a runtime. Pyodide gets the Python standard library and WebR gets base R; anything beyond that needs a Jupyter kernel with the packages already installed.

Failures that are the environment's fault rather than the code's, a runtime that is not installed, a driver that dies, a timeout, are never cached. A Python traceback is cached, because the code really does raise it.